Inteligência Artificial

Aula 06 – Parte 2: Aprendizado por Reforço

Parte 2

Prof. Dr. Danillo Roberto Pereira

Elaborado por: Gabriel Henrique Nunes dos Santos

Faculdade de Ciências e Tecnologia – FCT

Universidade Estadual Paulista (UNESP)

2026

Revisão: Q-learning tabular

Revisão: problema de aprendizado por reforço

- Continuamos em um **PDM**: estados, ações, transições e recompensas.
- O agente ainda busca uma política:

$$\pi(s) = \text{ação escolhida em } s.$$

- No aprendizado por reforço, o agente **não conhece** T nem R antecipadamente.

Ideia

O agente aprende uma política por interação: tenta ações, observa consequências e ajusta seus valores.

unesp

Q-learning tabular: vantagens e desvantagens

Vantagens

- Fácil de implementar.
- Tem garantia teórica de encontrar os q-valores adequados.

Desvantagens

- Seu uso é impraticável em problemas com muitos estados e/ou ações.
- Não permite que o agente aprenda a generalizar entre estados.

unesp

Limitação 1: tamanho do espaço de estados

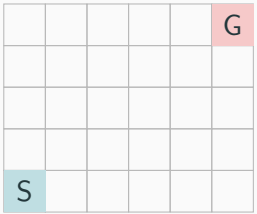
Tabela Q

Se há $|S|$ estados e $|A|$ ações, a tabela possui:

$$|S| \times |A|$$

entradas.

- Mundo grade pequeno: viável.
- Pac-Man, robôs, jogos e visão: número de configurações explode.
- Muitos pares estado-ação talvez nunca sejam visitados.



$6 \times 5 = 30$ estados

$\Rightarrow$ ainda cabe na tabela

Limitação 1: quando a tabela cresce demais

Exemplo conceitual

Considere um estado de jogo formado por:

$$\text{posição do agente} + \text{posição dos inimigos} + \text{itens restantes} + \text{direção atual}$$

- Cada elemento extra multiplica o número de configurações possíveis.
- O agente precisaria visitar muitos estados para aprender valores confiáveis.
- Mesmo que a memória suportasse a tabela, o treinamento poderia ser impraticável.

Consequência

A questão deixa de ser apenas "qual algoritmo usar?" e passa a ser "como representar o estado?".

Limitação 2: incapacidade de generalização

No tabular

Cada par (s, a) é aprendido separadamente.

- Saber que um estado é ruim não altera diretamente estados parecidos.
- O agente precisa repetir experiências similares muitas vezes.

Motivação

Queremos que o agente aprenda padrões: "fantasmas próximos tendem a ser ruins", "comida próxima tende a ser boa".

unesp

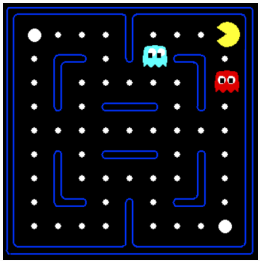
Limitação 2: exemplo

Q-learning tabular

Os estados são parecidos, mas mesmo que o agente descubra que o Estado A é ruim a tabela não diz nada sobre o Estado B. a tabela os trata como completamente independentes.



Estado A



Estado B

unesp

Q-learning com aproximação linear

Ideia: substituir a tabela por uma função

Q-learning tabular

$Q(s, a)$ vem de uma tabela

- Uma célula por par estado-ação.
- Pouca generalização.

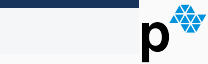
Q-learning com aprox. linear

$Q(s, a) \approx \widehat{Q}(s, a; \boldsymbol{w})$

- Uma função estima o valor.
- Estados parecidos podem compartilhar características, portanto terem valores parecidos.

Objetivo

Aprender os parâmetros da função, e não uma tabela gigantesca.



Vetor de coeficientes

Parâmetros aprendidos

No algoritmo Q-learning com aproximação linear, o objetivo é aprender um vetor de coeficientes $\boldsymbol{w} = (w_1, w_2, \ldots, w_n)$.
Uma vez aprendido, esse vetor pode ser usado para obter uma aproximação linear da Q-função

- Cada peso indica a importância de uma característica.
- Peso positivo: característica tende a aumentar o valor estimado.
- Peso negativo: característica tende a reduzir o valor estimado.

unesp

Características

Definição

Uma característica é uma função que transforma um par estado-ação em um número:

$$f_i : S \times A \to \mathbb{R}$$

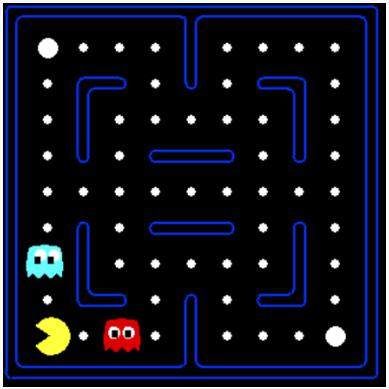
$$\boldsymbol{f}(s, a) = (f_1(s, a), f_2(s, a), \ldots, f_n(s, a))$$

- As características descrevem aspectos relevantes de (s, a).
- Podem ser binárias, contagens, distâncias etc.

unesp

Exemplo de características no Pac-Man

- $f_1(s, a)$: distância até a comida mais próxima após a ação.
- $f_2(s, a)$: distância até o fantasma mais próximo após a ação.
- $f_3(s, a)$: ação leva a capturar comida?
- $f_4(s, a)$: ação leva a estado terminal ruim?



Ponto importante

O agente não memoriza o labirinto inteiro.

Função Q

$$Q(s,a) = w_1 f_1(s,a) + w_2 f_2(s,a) + \cdots + w_n f_n(s,a)$$

$$Q(s,a) = \boldsymbol{w}^\top \boldsymbol{f}(s,a)$$

- As características são calculadas a partir do estado e da ação.
- Os pesos são ajustados com a experiência.
- Estados diferentes podem gerar vetores de características parecidos.

unesp

Exemplo: cálculo de Q

Pesos atuais

$$\boldsymbol{w} = (4, -1)$$

- $w_1 = 4$: comida próxima é boa.
- $w_2 = -1$: fantasma próximo é ruim.

Características da ação

$$\boldsymbol{f}(s, a) = (0{,}5, 1{,}0)$$

$$Q(s, a) = 4 \cdot 0{,}5 + (-1) \cdot 1{,}0 = 1$$

Leitura

A ação parece positiva, mas o valor ainda depende da qualidade dos pesos aprendidos.

unesp

Aprendizado: o que muda?

No tabular

Atualizamos diretamente uma célula:

$$Q(s, a) \leftarrow Q(s, a) + \alpha\delta$$

No aproximado linear

Não existe célula isolada. Atualizamos os pesos:

$$w_i \leftarrow w_i + \alpha\delta f_i(s, a)$$

- Se $f_i(s, a) = 0$, o peso w_i não é alterado nessa transição.
- Se $f_i(s, a)$ está ativa, ela influencia no novo peso.

unesp

Erro temporal no Q-learning aproximado

Alvo

$$y = r + \gamma \max_{a'} Q(s', a')$$

Predição atual

$$Q(s, a) = \boldsymbol{w}^\top \boldsymbol{f}(s, a)$$

Erro temporal

$$\delta = y - Q(s, a)$$

unesp

Algoritmo: Q-learning com aproximação linear

1. Inicialize os pesos w_i; por exemplo, todos com zero.
2. Observe o estado atual s.
3. Escolha uma ação a usando uma estratégia de exploração.
4. Execute a e observe r e s'.
5. Calcule:
$$\delta = r + \gamma \max_{a'} Q(s', a') - Q(s, a)$$
6. Para cada característica i:
$$w_i \leftarrow w_i + \alpha \delta f_i(s, a)$$
7. Atualize $s \leftarrow s'$ e continue.

unesp

Exemplo: atualização dos pesos

Antes da experiência

$$\boldsymbol{w} = (4, -1), \quad \boldsymbol{f}(s, a) = (0{,}5, 1)$$

$$Q(s, a) = 1$$

Experiência

$$r = -500, \quad s' \text{ terminal}$$

$$\max_{a'} Q(s', a') = 0, \quad \gamma = 0{,}9$$

Erro temporal

$$y = -500 + 0 = -500$$

$$\delta = -500 - 1 = -501$$

Atualização

$$w_i \leftarrow w_i + \alpha \delta f_i(s, a)$$

Com α pequeno, os pesos associados às características ativas são reduzidos na direção correta.

Exemplo: interpretação da atualização

O que aconteceu?

A ação parecia razoável, mas levou a uma recompensa muito negativa.

- O agente superestimou $Q(s, a)$.
- O erro temporal foi negativo.
- Características ativas nesse estado-ação foram penalizadas.
- Em experiências futuras, ações com características parecidas tendem a receber valor menor.

Generalização

A atualização não corrige apenas um estado específico; ela altera a avaliação de todos os estados-ações que compartilham características semelhantes.

Análise: vantagens e desvantagens

Vantagens

- Usa menos memória do que a tabela.
- Generaliza para estados não visitados.
- Permite aprendizado em espaços grandes.

Desvantagens

- Depende de boas características.
- Pode confundir estados diferentes com características parecidas.
- Fora do caso tabular, as garantias são mais frágeis.

Cuidado

Aproximar bem os valores Q não é exatamente o mesmo que escolher boas ações.

unesp

Q-learning e mínimos quadrados

Q-learning e mínimos quadrados

Ideia

O Q-learning aproximado pode ser visto como uma forma incremental de regressão: cada experiência produz um alvo e a função tenta aproximá-lo.

- Entrada: vetor de características $\boldsymbol{f}(s, a)$.
- Saída prevista: $Q(s, a)$.
- Alvo: $r + \gamma \max_{a'} Q(s', a')$.
- Erro: alvo menos predição.

unesp

Aproximação linear como regressão

Regressão linear

$$\hat{y} = \boldsymbol{w}^\top \boldsymbol{x}$$

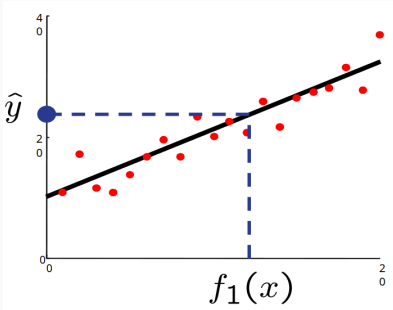
Q-learning com função linear

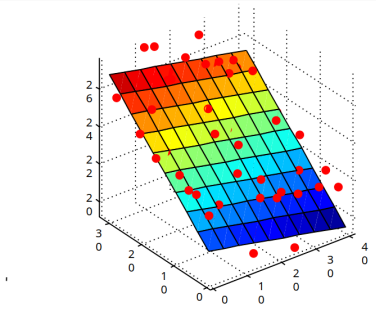
$$Q(s, a) = \boldsymbol{w}^\top \boldsymbol{f}(s, a)$$

- Na regressão comum, ajustamos $\hat{y}$ para aproximar y.
- No Q-learning, ajustamos Q para aproximar um alvo temporal.
- Cada transição observada é uma amostra de treinamento.

unesp

Aproximação linear como regressão





Predição (em 1D):

$$\widehat{y} = w_0 + w_1 f_1(x)$$

Predição (em 2D):

$$\widehat{y}_i = w_0 + w_1 f_1(x) + w_2 f_2(x)$$

unesp

Q-learning e mínimos quadrados

Ideia central

No Q-learning com aproximação linear, o valor de um estado-ação é estimado por:

$$Q(s, a) = \mathbf{w}^\top \mathbf{f}(s, a)$$

O aprendizado consiste em ajustar os pesos $\mathbf{w}$ para produzir estimativas cada vez melhores.

Experiência $\longrightarrow$ Alvo $\longrightarrow$ Erro $\longrightarrow$ Atualização dos pesos

unesp

Qual valor queremos aproximar?

Ao observar uma transição (s, a, r, s'), o Q-learning constrói um novo valor de referência para $Q(s, a)$:

$$\boxed{y = r + \gamma \max_{a'} Q(s', a')}$$

Interpretação

Esse valor combina:

- a recompensa recebida agora, r;
- a melhor recompensa futura estimada a partir de s'.

Chamaremos y de **alvo temporal**.

unesp

Alvo e predição

Para a mesma experiência, temos dois valores:

Predição atual

$$Q(s, a) = \mathbf{w}^\top \mathbf{f}(s, a)$$

É o que o agente **estimava** antes de observar o resultado.

Novo alvo

$$y = r + \gamma \max_{a'} Q(s', a')$$

É o que a nova experiência **sugere** para esse valor.

O aprendizado deve aproximar a **predição** do **alvo**.

unesp

Erro da aproximação

A diferença entre o alvo e a predição é:

$$\boxed{\delta = y - Q(s, a)}$$

Substituindo o alvo:

$$\delta = r + \gamma \max_{a'} Q(s', a') - Q(s, a)$$

Leitura intuitiva

- $\delta > 0$: o valor estava sendo **subestimado**.
- $\delta < 0$: o valor estava sendo **superestimado**.
- $\delta \approx 0$: predição e alvo estão próximos.

Por que mínimos quadrados?

Queremos reduzir a diferença entre o alvo e a predição. Uma forma de medir esse erro é:

$$\boxed{\ell(\mathbf{w}) = \frac{1}{2}\left[y - Q(s,a)\right]^2}$$

Como

$$\delta = y - Q(s,a),$$

também podemos escrever:

$$\ell(\mathbf{w}) = \frac{1}{2}\delta^2$$

Por que elevar ao quadrado?

Erros positivos e negativos não se cancelam, e erros maiores recebem uma penalização maior.

Minimizando o erro

Como

$$Q(s, a) = \mathbf{w}^\top \mathbf{f}(s, a),$$

o erro depende diretamente dos pesos $\mathbf{w}$. O objetivo do aprendizado é ajustar:

$$\mathbf{w}$$

de forma que:

$$Q(s, a) \longrightarrow r + \gamma \max_{a'} Q(s', a')$$

Objetivo

Mover os pesos na direção que reduz o erro entre a predição atual e o alvo produzido pela experiência.

Por que derivada parcial?

No Q-learning aproximado:

$$Q(s, a) = \sum_j w_j f_j(s, a)$$

O erro depende de vários pesos: $\mathbf{w} = (w_1, w_2, \ldots, w_n)$

Pergunta

Como o erro muda quando alteramos apenas um peso w_i?

$$\frac{\partial \ell}{\partial w_i}$$

Interpretação

A derivada parcial mede a influência do peso w_i no erro.



Derivando o erro

Partimos de:

$$\ell(\mathbf{w}) = \frac{1}{2}\left[y - Q(s,a)\right]^2$$

Definindo:

$$\delta = y - Q(s,a)$$

temos:

$$\ell(\mathbf{w}) = \frac{1}{2}\delta^2$$

Logo, pela regra da cadeia:

$$\frac{\partial \ell}{\partial w_i} = \delta \frac{\partial \delta}{\partial w_i}$$

unesp

Derivada da predição

Como

$$\delta = y - Q(s, a)$$

e

$$Q(s, a) = \sum_j w_j f_j(s, a),$$

então:

$$\frac{\partial \delta}{\partial w_i} = -\frac{\partial Q(s, a)}{\partial w_i}$$

Como apenas o termo $w_i f_i(s, a)$ depende de w_i:

$$\frac{\partial Q(s, a)}{\partial w_i} = f_i(s, a)$$

Portanto:

$$\frac{\partial \delta}{\partial w_i} = -f_i(s, a)$$

unesp

Gradiente do erro

Juntando os resultados:

$$\frac{\partial \ell}{\partial w_i} = \delta \frac{\partial \delta}{\partial w_i}$$

e

$$\frac{\partial \delta}{\partial w_i} = -f_i(s, a)$$

obtemos:

$$\boxed{\frac{\partial \ell}{\partial w_i} = -\delta f_i(s, a)}$$

Leitura intuitiva

A característica $f_i(s, a)$ determina quanto o peso w_i participa da correção do erro.

Atualização dos pesos

Para reduzir o erro, movemos os pesos na direção oposta ao gradiente:

$$w_i \leftarrow w_i - \alpha \frac{\partial \ell}{\partial w_i}$$

Como

$$\frac{\partial \ell}{\partial w_i} = -\delta f_i(s, a),$$

temos:

$$\boxed{w_i \leftarrow w_i + \alpha \delta f_i(s, a)}$$

unesp

Atualização no Q-learning linear

No Q-learning:

$$\delta = r + \gamma \max_{a'} Q(s', a') - Q(s, a)$$

Substituindo na atualização:

$$\boxed{w_i \leftarrow w_i + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right] f_i(s, a)}$$

Conclusão

A regra de atualização dos pesos vem da minimização do erro quadrático entre o alvo temporal e a predição atual.

unesp

Interpretação da atualização

Considere:

$$w_i \leftarrow w_i + \alpha\, \delta\, f_i(s, a)$$

$\delta > 0$

O resultado foi **melhor** que o previsto.

Os pesos das características ativas tendem a aumentar.

$\delta < 0$

O resultado foi **pior** que o previsto.

Os pesos das características ativas tendem a diminuir.

Caso especial

Se $f_i(s, a) = 0$, aquela característica não participa da atualização.

Exploração versus aproveitamento

Exploração versus aproveitamento

Dilema central

Durante o treinamento, o agente quer escolher boas ações, mas ainda não sabe quais ações são realmente boas.

Exploração

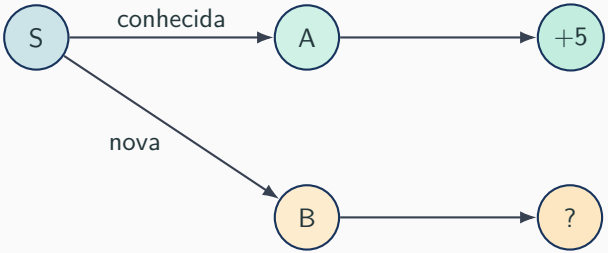
Escolher ações ainda pouco conhecidas para obter informação.

Aproveitamento

Escolher a ação que parece melhor com o conhecimento atual.

unesp

Exemplo: duas rotas possíveis



Discussão

A rota conhecida dá recompensa razoável. A rota nova pode ser melhor, igual ou desastrosa. O agente precisa decidir quando vale pagar o custo de descobrir.

unesp

De que forma explorar?

Estratégias comuns

- Ações aleatórias ocasionais.
- Inicialização otimista dos valores.
- Funções de exploração com bônus para incerteza.
- Amostragem baseada em incerteza ou contagem de visitas.

Nesta aula

Vamos focar em duas ideias didáticas: ϵ-greedy e funções de exploração.

unesp

ϵ-greedy

Regra

A cada decisão:

$$\begin{cases} \text{ação aleatória}, & \text{com probabilidade } \epsilon, \\ \arg\max_a Q(s,a), & \text{com probabilidade } 1-\epsilon. \end{cases}$$

- Simples e muito usada como linha de base.
- Garante alguma exploração se $\epsilon > 0$.
- Pode continuar tomando ações ruins mesmo depois de aprender.

unesp

Exemplo: ϵ-greedy

Suponha $\epsilon = 0{,}1$

Em média, a cada 100 decisões:

10 ações aleatórias e 90 ações gulosas.

Início do treino

Exploração ajuda porque os valores ainda são pouco confiáveis.

Fim do treino

Exploração constante pode prejudicar o desempenho.

Solução prática

Usar decaimento: ϵ alto no início e menor ao longo do treinamento.

unesp

Funções de exploração

Ideia

Em vez de escolher ações aleatórias, tornar ações pouco visitadas temporariamente mais atraentes.

$$f(u, n) = u + \frac{k}{n+1}$$

- u: utilidade estimada.
- n: número de visitas ao estado-ação.
- k: intensidade do bônus de exploração.

Intuição

Com poucas visitas, o bônus é grande. Com muitas visitas, o bônus desaparece.

Exemplo: bônus de exploração

Ação conhecida

$$u = 8, \quad n = 20, \quad k = 10$$

$$f(8, 20) = 8 + \frac{10}{21} \approx 8{,}48$$

Ação pouco visitada

$$u = 6, \quad n = 1, \quad k = 10$$

$$f(6, 1) = 6 + \frac{10}{2} = 11$$

Resultado

Mesmo com valor estimado menor, a ação pouco visitada pode ser escolhida por ainda ser incerta.

unesp

Arrependimento

Definição intuitiva

Arrependimento mede o custo acumulado das escolhas subótimas durante o aprendizado.

$$\text{Regret}(T) = \sum_{t=1}^{T} (r_t^\star - r_t)$$

- $r_t^\star$: recompensa esperada da melhor ação possível naquele momento.
- r_t: recompensa obtida pela ação escolhida.
- Mesmo aprendendo a política ótima no final, o agente erra ao longo do caminho.

unesp

Arrependimento: leitura prática

Exploração aleatória

Pode descobrir boas ações, mas desperdiça muitas decisões em ações ruins.

Exploração dirigida

Tenta explorar onde ainda existe incerteza relevante.

Não basta aprender a ser ótimo. O agente deve tentar aprender com o menor custo possível.

unesp

Policy search

Limitação da abordagem baseada em valor

Problema

Às vezes, a função Q pode ser uma estimativa ruim dos retornos absolutos, mas ainda assim ordenar bem as ações.

- Para predizer retornos, queremos valores numericamente precisos.
- Para agir, basta escolher a melhor ação.
- O objetivo final do agente é maximizar recompensas, não necessariamente modelar Q perfeitamente.

unesp

Policy search

Ideia

Aprender diretamente uma política que maximize recompensas, em vez de primeiro aprender uma função de valor precisa.

$$\pi_\theta(s) = \text{política parametrizada por } \theta$$

- Começa-se com uma política inicial.
- Ajustam-se os parâmetros da política.
- Avalia-se se o retorno médio melhorou.

unesp

Policy search: busca local nos parâmetros

1. Comece com uma política inicial, por exemplo obtida com Q-learning.
2. Altere levemente um peso ou parâmetro.
3. Execute vários episódios para estimar o retorno médio.
4. Mantenha a alteração se a política melhorar.
5. Repita o processo.

Custo

Avaliar uma política exige muitos episódios; com muitos parâmetros, a busca ingênua se torna cara.

unesp

Referências

- Sutton, R. S.; Barto, A. G. *Reinforcement Learning: An Introduction*. 2nd ed.
- CS188 – Introduction to Artificial Intelligence, UC Berkeley.

unesp